

October 4, 2026

```
[1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

```
[2]: df = pd.read_csv("heart.csv")
```

```
[3]: df.head()
```

```
[3]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

```
[4]: #EDA
df.columns
```

```
[4]: Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
          'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
          'HeartDisease'],
          dtype='str')
```

```
[5]: df.shape
```

```
[5]: (918, 12)
```

```
[6]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                    918 non-null    int64
1   Sex                    918 non-null    str
2   ChestPainType          918 non-null    str
3   RestingBP              918 non-null    int64
4   Cholesterol             918 non-null    int64
5   FastingBS              918 non-null    int64
6   RestingECG             918 non-null    str
7   MaxHR                  918 non-null    int64
8   ExerciseAngina         918 non-null    str
9   Oldpeak                918 non-null    float64
10  ST_Slope               918 non-null    str
11  HeartDisease           918 non-null    int64
dtypes: float64(1), int64(6), str(5)
memory usage: 97.6 KB

```

```
[7]: df.describe()
```

```

[7]:
      count  Age  RestingBP  Cholesterol  FastingBS  MaxHR  \
count  918.000000  918.000000  918.000000  918.000000  918.000000
mean    53.510893  132.396514  198.799564    0.233115  136.809368
std      9.432617   18.514154  109.384145    0.423046   25.460334
min     28.000000    0.000000    0.000000    0.000000   60.000000
25%     47.000000  120.000000  173.250000    0.000000  120.000000
50%     54.000000  130.000000  223.000000    0.000000  138.000000
75%     60.000000  140.000000  267.000000    0.000000  156.000000
max     77.000000  200.000000  603.000000    1.000000  202.000000

      Oldpeak  HeartDisease
count  918.000000  918.000000
mean     0.887364    0.553377
std     1.066570    0.497414
min     -2.600000    0.000000
25%      0.000000    0.000000
50%      0.600000    1.000000
75%      1.500000    1.000000
max      6.200000    1.000000

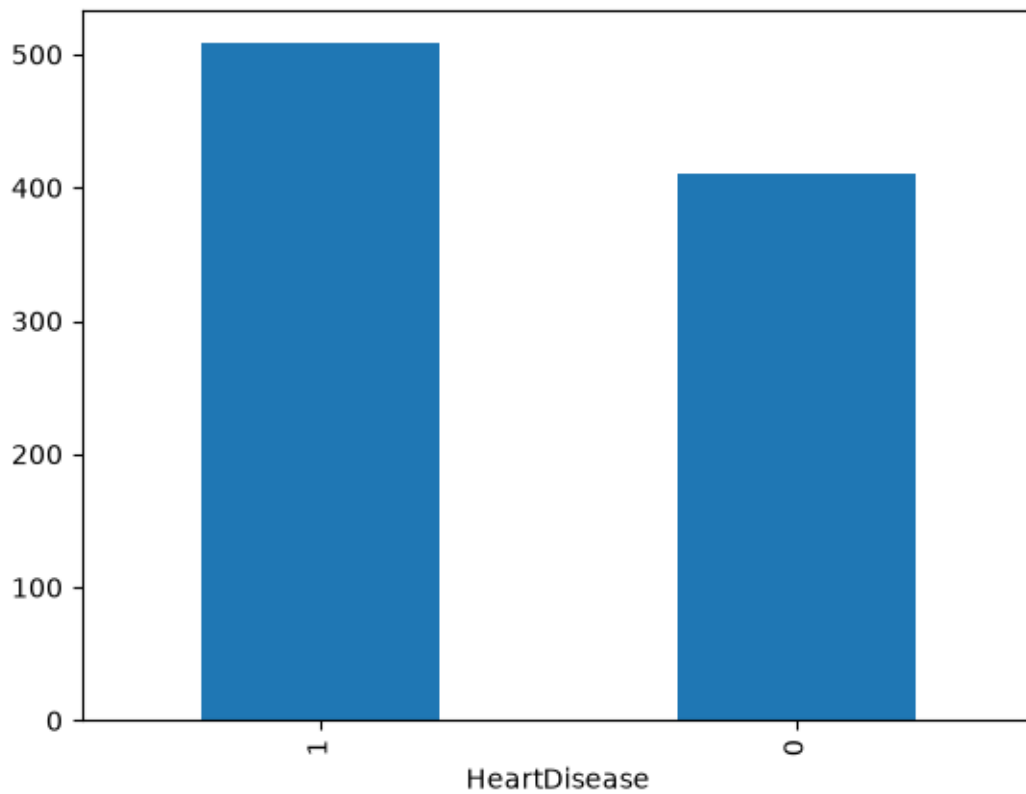
```

```
[8]: df.duplicated().sum()
```

```
[8]: np.int64(0)
```

```
[9]: df['HeartDisease'].value_counts().plot(kind="bar")
```

```
[9]: <Axes: xlabel='HeartDisease'>
```



```
[10]: df.isnull().sum()
```

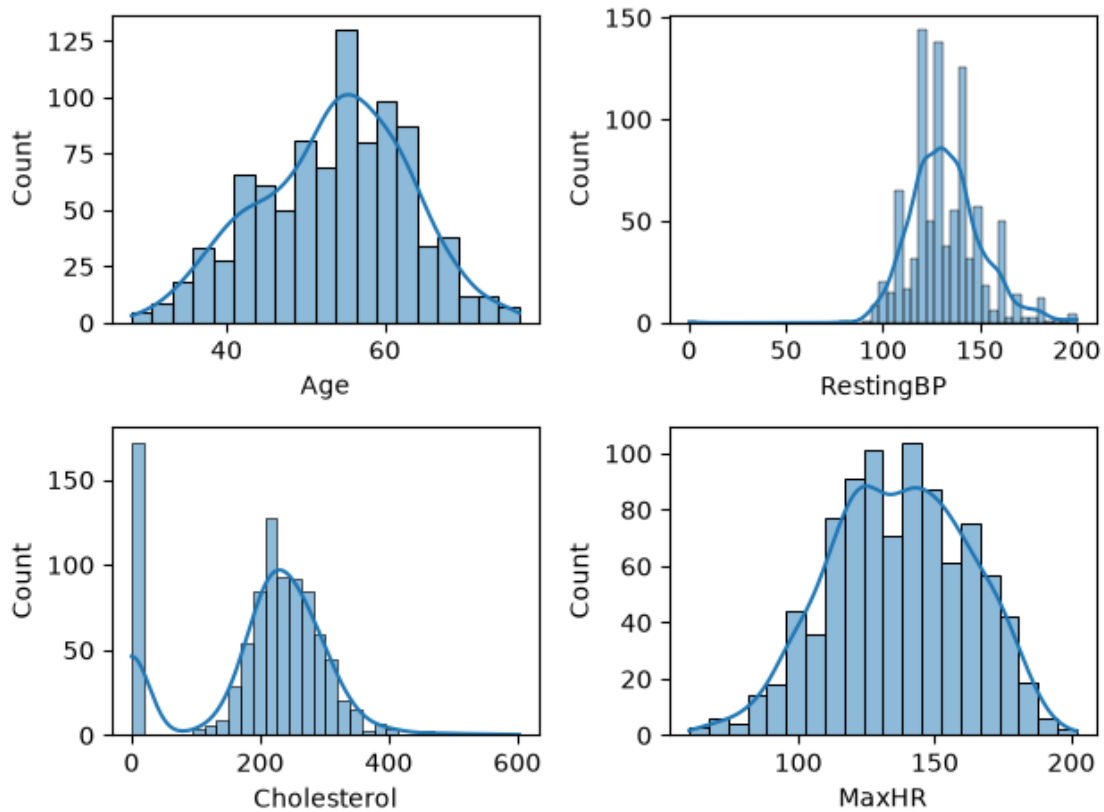
```
[10]: Age          0
      Sex          0
      ChestPainType  0
      RestingBP     0
      Cholesterol   0
      FastingBS     0
      RestingECG    0
      MaxHR        0
      ExerciseAngina 0
      Oldpeak       0
      ST_Slope      0
      HeartDisease  0
      dtype: int64
```

```
[11]: def plotting(var,num):
      plt.subplot(2,2,num)
      sns.histplot(df[var],kde=True)
```

```

plotting('Age',1)
plotting('RestingBP',2)
plotting('Cholesterol',3)
plotting('MaxHR',4)
plt.tight_layout()

```



```

[12]: ch_mean = df.loc[df['Cholesterol'] != 0, 'Cholesterol'].mean()
      ch_mean

```

```

[12]: np.float64(244.6353887399464)

```

```

[13]: df['Cholesterol'] = df['Cholesterol'].replace(0, ch_mean)
      df['Cholesterol'] = df['Cholesterol'].round(2)

```

```

[14]: resting_bp_mean = df.loc[df['RestingBP'] != 0, 'RestingBP'].mean()

      df['RestingBP'] = df['RestingBP'].replace(0, resting_bp_mean)

      df['RestingBP'] = df['RestingBP'].round(2)

```

```
[15]: import sheryanalysis as sh
      sh.analyze(df)
```

Basic Analysis Report

```
Shape: (918, 12)
Columns: ['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol',
'FastingBS', 'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
'HeartDisease']
```

No null values found

```
Categorical Columns: ['Sex', 'ChestPainType', 'FastingBS', 'RestingECG',
'ExerciseAngina', 'ST_Slope', 'HeartDisease']
```

```
Numerical Columns: ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']
```

```
[15]: {'shape': (918, 12),
      'columns': ['Age',
        'Sex',
        'ChestPainType',
        'RestingBP',
        'Cholesterol',
        'FastingBS',
        'RestingECG',
        'MaxHR',
        'ExerciseAngina',
        'Oldpeak',
        'ST_Slope',
        'HeartDisease'],
      'dtypes': {'Age': dtype('int64'),
        'Sex': <StringDtype(na_value=nan)>,
        'ChestPainType': <StringDtype(na_value=nan)>,
        'RestingBP': dtype('float64'),
        'Cholesterol': dtype('float64'),
        'FastingBS': dtype('int64'),
        'RestingECG': <StringDtype(na_value=nan)>,
        'MaxHR': dtype('int64'),
        'ExerciseAngina': <StringDtype(na_value=nan)>,
        'Oldpeak': dtype('float64'),
        'ST_Slope': <StringDtype(na_value=nan)>,
        'HeartDisease': dtype('int64')},
      'null_counts': {'Age': 0,
        'Sex': 0,
        'ChestPainType': 0,
        'RestingBP': 0,
        'Cholesterol': 0,
```

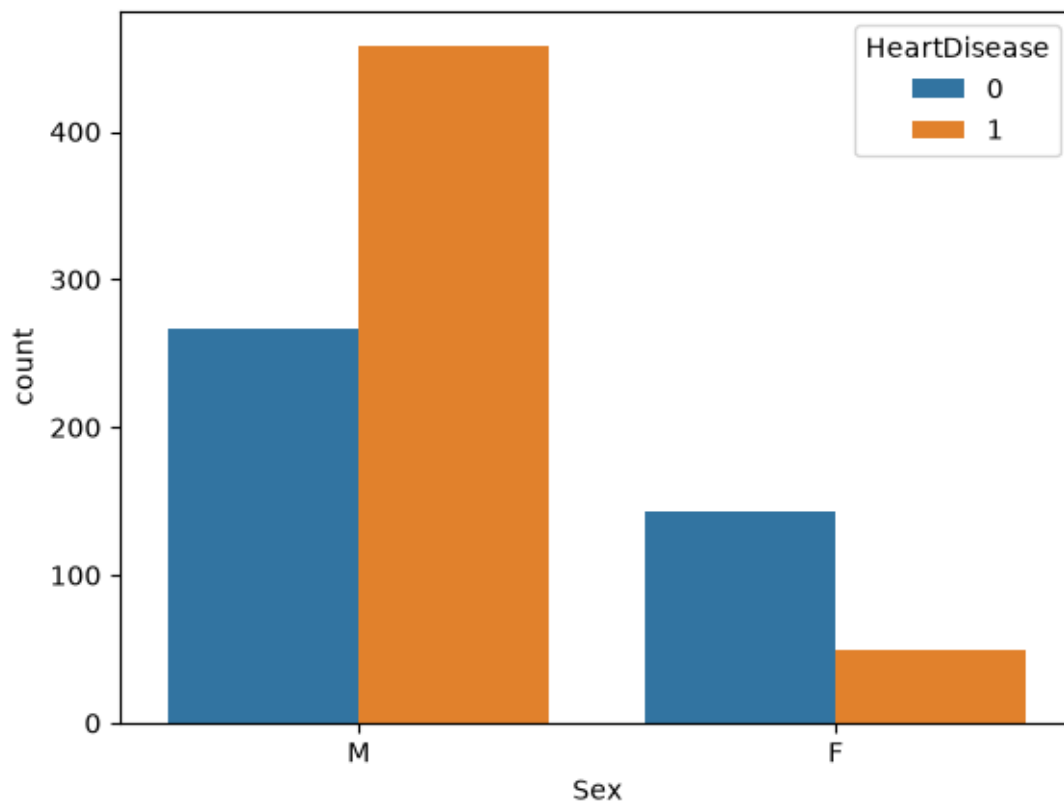
```

'FastingBS': 0,
'RestingECG': 0,
'MaxHR': 0,
'ExerciseAngina': 0,
'Oldpeak': 0,
'ST_Slope': 0,
'HeartDisease': 0},
'total_rows': 918,
'column_types': {'categorical': ['Sex',
    'ChestPainType',
    'FastingBS',
    'RestingECG',
    'ExerciseAngina',
    'ST_Slope',
    'HeartDisease'],
    'numerical': ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak'],
    'datetime': [],
    'text': []}}

```

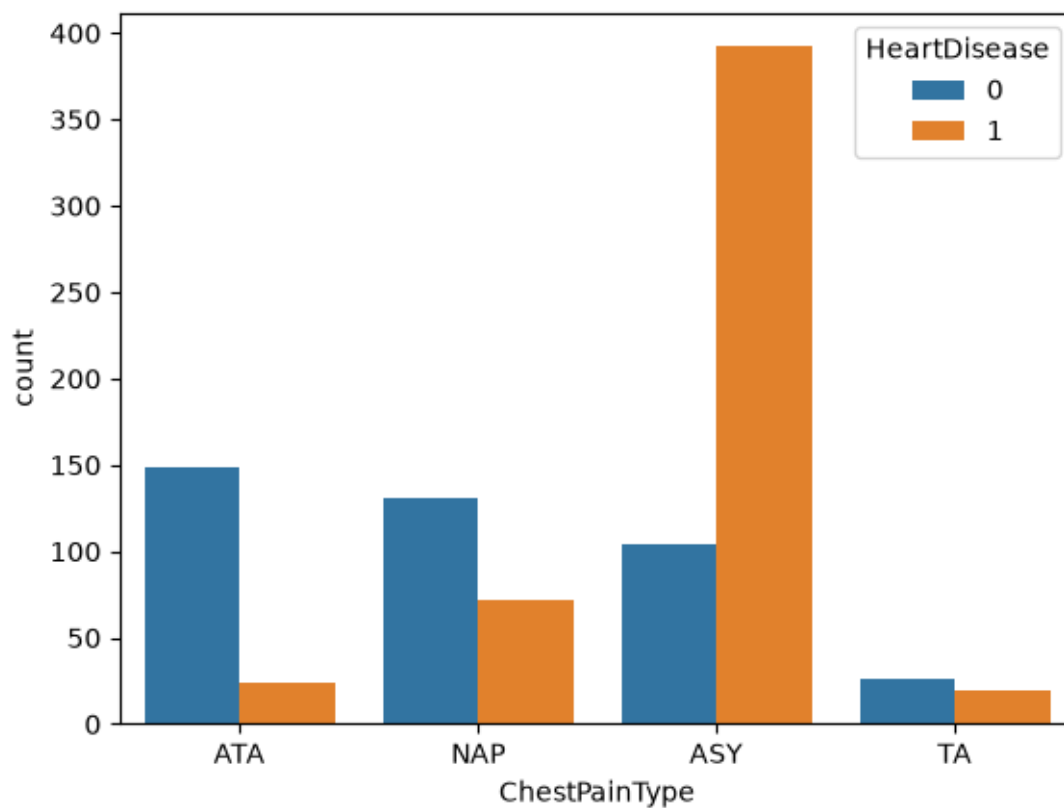
```
[16]: sns.countplot(x=df['Sex'],hue =df ['HeartDisease'])
```

```
[16]: <Axes: xlabel='Sex', ylabel='count'>
```



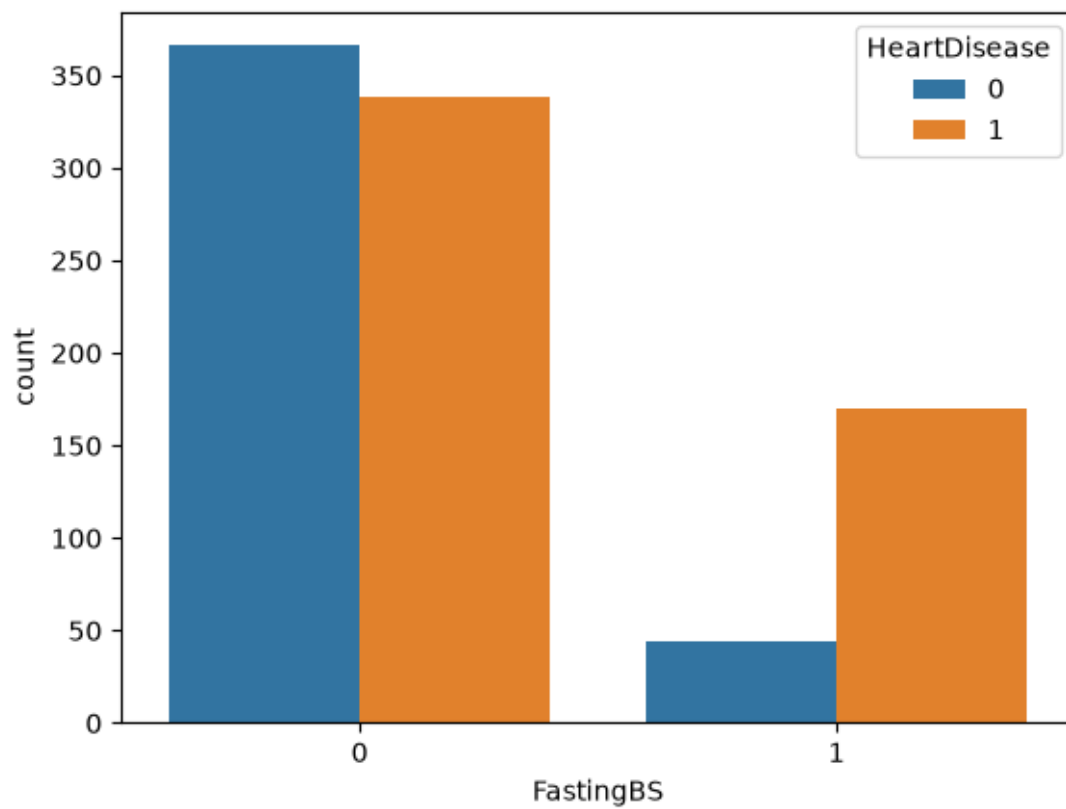
```
[17]: sns.countplot(x=df['ChestPainType'],hue =df ['HeartDisease'])
```

```
[17]: <Axes: xlabel='ChestPainType', ylabel='count'>
```



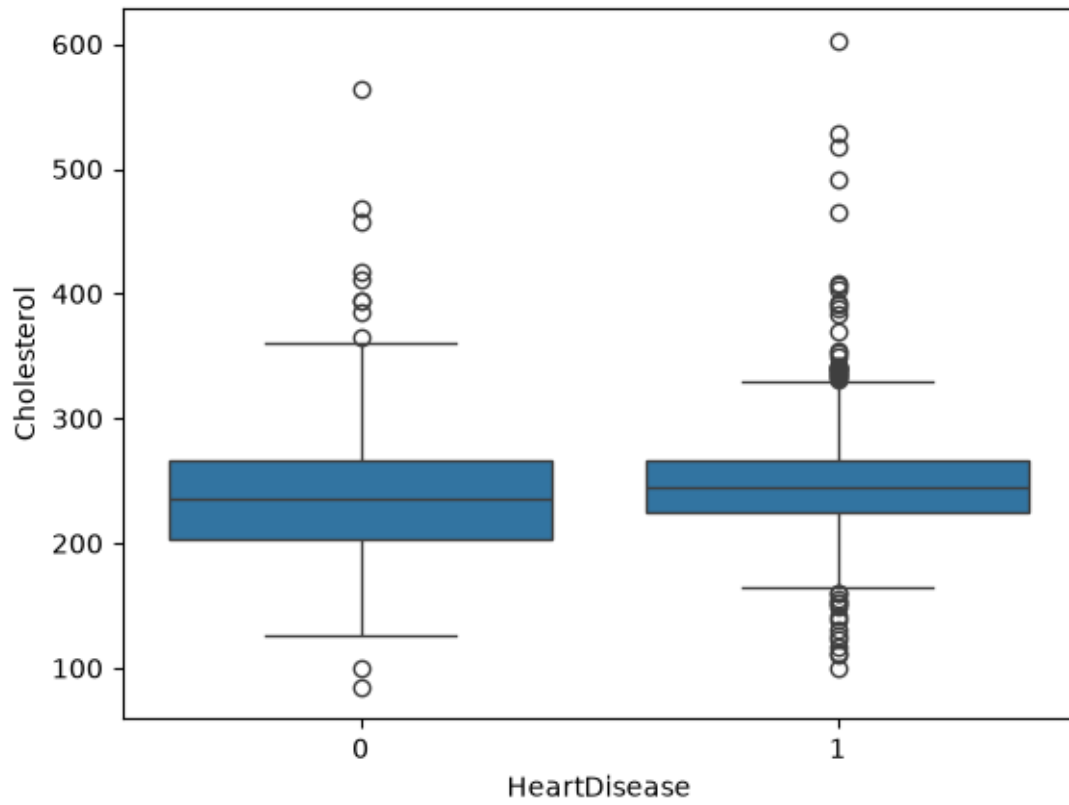
```
[18]: sns.countplot(x=df['FastingBS'],hue =df ['HeartDisease'])
```

```
[18]: <Axes: xlabel='FastingBS', ylabel='count'>
```



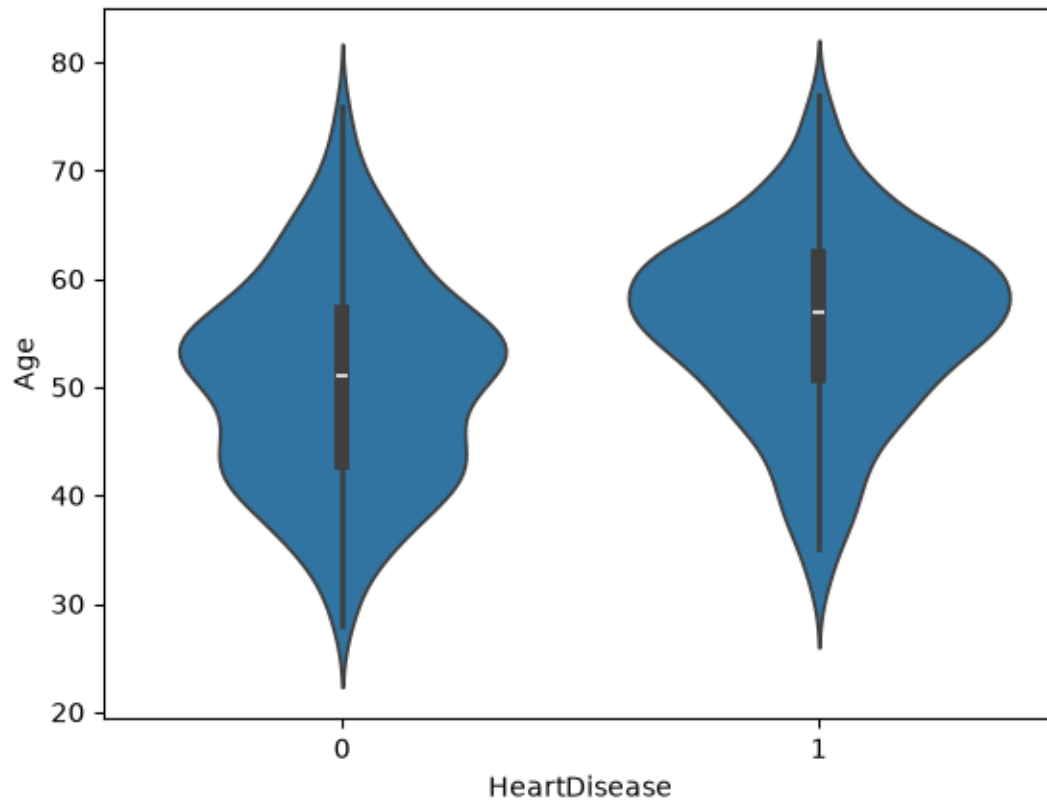
```
[19]: sns.boxplot(x='HeartDisease',y='Cholesterol', data=df)
```

```
[19]: <Axes: xlabel='HeartDisease', ylabel='Cholesterol'>
```



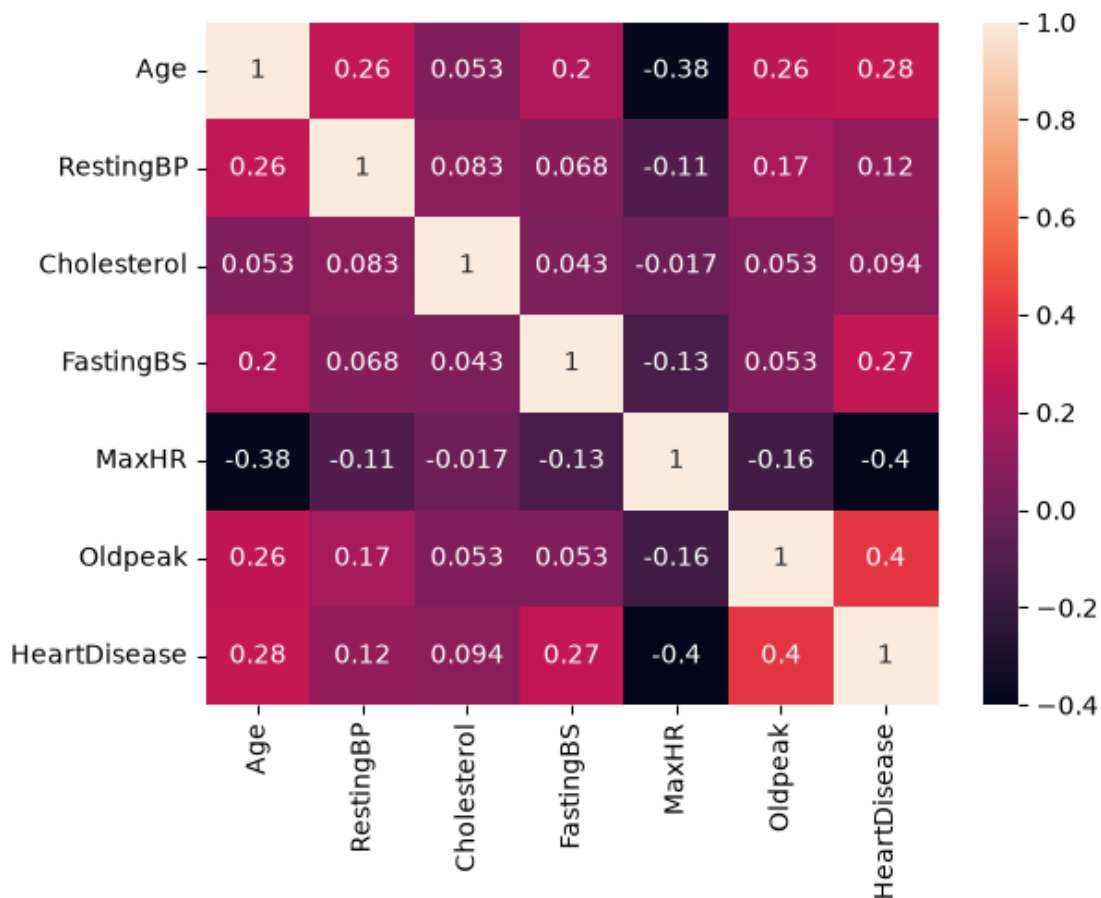
```
[20]: sns.violinplot(x='HeartDisease', y='Age', data=df)
```

```
[20]: <Axes: xlabel='HeartDisease', ylabel='Age'>
```



```
[21]: sns.heatmap(df.corr(numeric_only=True), annot=True)
```

```
[21]: <Axes: >
```



```
[22]: #data preprocessing and cleaning
```

```
[23]: df.head()
```

```
[23]:
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140.0	289.0	0	Normal	172	
1	49	F	NAP	160.0	180.0	0	Normal	156	
2	37	M	ATA	130.0	283.0	0	ST	98	
3	48	F	ASY	138.0	214.0	0	Normal	108	
4	54	M	NAP	150.0	195.0	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

```
[24]: df_encoded = pd.get_dummies(df, drop_first=True)
```

```
[25]: df_encoded
```

```
[25]:
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease	\
0	40	140.0	289.0	0	172	0.0	0	
1	49	160.0	180.0	0	156	1.0	1	
2	37	130.0	283.0	0	98	0.0	0	
3	48	138.0	214.0	0	108	1.5	1	
4	54	150.0	195.0	0	122	0.0	0	
..	...	...	...	...	...	...	...	
913	45	110.0	264.0	0	132	1.2	1	
914	68	144.0	193.0	1	141	3.4	1	
915	57	130.0	131.0	0	115	1.2	1	
916	57	130.0	236.0	0	174	0.0	1	
917	38	138.0	175.0	0	173	0.0	0	

	Sex_M	ChestPainType_ATA	ChestPainType_NAP	ChestPainType_TA	\
0	True	True	False	False	
1	False	False	True	False	
2	True	True	False	False	
3	False	False	False	False	
4	True	False	True	False	
..	...	...	...	...	
913	True	False	False	True	
914	True	False	False	False	
915	True	False	False	False	
916	False	True	False	False	
917	True	False	True	False	

	RestingECG_Normal	RestingECG_ST	ExerciseAngina_Y	ST_Slope_Flat	\
0	True	False	False	False	
1	True	False	False	True	
2	False	True	False	False	
3	True	False	True	True	
4	True	False	False	False	
..	...	...	...	...	
913	True	False	False	True	
914	True	False	False	True	
915	True	False	True	True	
916	False	False	False	True	
917	True	False	False	False	

	ST_Slope_Up
0	True
1	False
2	True

```

3      False
4      True
..      ...
913    False
914    False
915    False
916    False
917    True

```

[918 rows x 16 columns]

```
[26]: df_encoded = df_encoded.astype(int)
```

```
[27]: df_encoded
```

```
[27]:
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease	\
0	40	140	289	0	172	0	0	
1	49	160	180	0	156	1	1	
2	37	130	283	0	98	0	0	
3	48	138	214	0	108	1	1	
4	54	150	195	0	122	0	0	
..	...	...	...	...	...	...	...	
913	45	110	264	0	132	1	1	
914	68	144	193	1	141	3	1	
915	57	130	131	0	115	1	1	
916	57	130	236	0	174	0	1	
917	38	138	175	0	173	0	0	

	Sex_M	ChestPainType_ATA	ChestPainType_NAP	ChestPainType_TA	\
0	1	1	0	0	
1	0	0	1	0	
2	1	1	0	0	
3	0	0	0	0	
4	1	0	1	0	
..	...	...	...	...	
913	1	0	0	1	
914	1	0	0	0	
915	1	0	0	0	
916	0	1	0	0	
917	1	0	1	0	

	RestingECG_Normal	RestingECG_ST	ExerciseAngina_Y	ST_Slope_Flat	\
0	1	0	0	0	
1	1	0	0	1	
2	0	1	0	0	
3	1	0	1	1	
4	1	0	0	0	

```

..          ...          ...          ...          ...
913          1          0          0          1
914          1          0          0          1
915          1          0          1          1
916          0          0          0          1
917          1          0          0          0

```

```

      ST_Slope_Up
0          1
1          0
2          1
3          0
4          1
..          ...
913         0
914         0
915         0
916         0
917         1

```

[918 rows x 16 columns]

```
[28]: df_encoded.columns
```

```
[28]: Index(['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak',
          'HeartDisease', 'Sex_M', 'ChestPainType_ATA', 'ChestPainType_NAP',
          'ChestPainType_TA', 'RestingECG_Normal', 'RestingECG_ST',
          'ExerciseAngina_Y', 'ST_Slope_Flat', 'ST_Slope_Up'],
          dtype='str')
```

```
[29]: from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
      from sklearn.metrics import accuracy_score, f1_score, classification_report
      from sklearn.linear_model import LogisticRegression
      from sklearn.naive_bayes import GaussianNB
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.svm import SVC
      from sklearn.neighbors import KNeighborsClassifier
```

```
[30]: X = df_encoded.drop('HeartDisease', axis=1)
      y = df_encoded['HeartDisease']
```

```
[31]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, stratify=y, test_size=0.2, random_state=42
      )
```

```
[32]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
[33]: models = {
    "Logistic Regression": LogisticRegression(),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(),
    "SVM (RBF Kernel)": SVC(probability=True)
}
```

```
[34]: results = []
```

```
[35]: for name, model in models.items():
    model.fit(X_train_scaled, y_train)

    y_pred = model.predict(X_test_scaled)

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    results.append({
        'Model': name,
        'Accuracy': round(acc, 4),
        'F1 Score': round(f1, 4)
    })
```

```
[36]: results
```

```
[36]: [{'Model': 'Logistic Regression', 'Accuracy': 0.875, 'F1 Score': 0.8878},
      {'Model': 'KNN', 'Accuracy': 0.8859, 'F1 Score': 0.8986},
      {'Model': 'Naive Bayes', 'Accuracy': 0.8696, 'F1 Score': 0.8788},
      {'Model': 'Decision Tree', 'Accuracy': 0.7609, 'F1 Score': 0.7755},
      {'Model': 'SVM (RBF Kernel)', 'Accuracy': 0.8641, 'F1 Score': 0.8804}]
```

```
[37]: import joblib

joblib.dump(models['KNN'], 'KNN_heart.pkl')
joblib.dump(scaler, 'scaler.pkl')
joblib.dump(X.columns.tolist(), 'columns.pkl')
```

```
[37]: ['columns.pkl']
```

```
[ ]:
```